

15LSS_sample_gas IOC: implementation specification

Document	IOC specification for handoff to an implementing agent
Date	2026-09-25
Repository	%USERPROFILE%\Documents\Claude Locals\o2-purge (git; commit identity in §2.4)
Status	Approved behaviour, frozen in simulator tag <code>sim-v1.0</code> . Not yet implemented.
Audience	An agent with no prior context . Read §0–§3 before touching anything.

This document says **what** the IOC must do, and every name, number and rule it must use. It does not prescribe code, but §12 gives recommended structure. Where this document is silent, the reference implementation decides (§0.2).

0. How to use this document

0.1 Normative sources, in order of precedence

1. **This document.**
2. **The reference implementation:** the `Controller` class in `simulator/sample_gas_simulator.html` at git tag `sim-v1.0`.
 - It is a working, tested model of this exact controller.
 - For any behaviour not fully specified here, reproduce what `Controller` does, including message texts.
 - The key methods are listed in the traceability table (§17).
3. **The design spec** `docs/superpowers/specs/2026-09-24-o2-purge-feedback-design.md`, which gives the reasons behind each decision and the measured physics (§2.1).
4. **The analysis** in `analysis/` (README and `run_all.py`): the measured data behind the numbers.

If sources 1 and 2 disagree, **stop and ask the user**; do not pick one silently. The simulator guides in `docs/simulator/` (user guide, illustrated tour, agent guide) are recommended reading.

0.2 Run the reference implementation

- **Open** `simulator/sample_gas_simulator.html` in a browser.
- **Self-test: ? Help** → **Self-test** runs all 19 scenarios in ~10 s.
- **Headless use:** `window.SIM` offers `run(station, seconds)`, `scenario(station, index, seconds) → {state, alarms, log}`, and `selfTest(onRow, onDone)`.
- **Golden vectors:** use `SIM` to generate expected outputs for your tests (§14.4).

0.3 Terms

- **Station:** one controller instance (15IDC or 15IDE). One IOC runs several.
 - **Tick:** the controller's 1 Hz cycle.
 - **Mode:** the enclosure configuration (lid type): A = normal lid, B = collimator lid, C and D spare.
 - **m%:** 0.001 % O₂.
 - **SLPM:** standard litres per minute of helium.
-

1. Mission and scope

- **What it does:** implement the IOC `15LSS_sample_gas`, which controls the helium purge and O2 concentration of a sample enclosure at APS 15-ID. It drives an Alicat BASIS mass flow controller from an O2 analyzer reading, with one independent controller per end station.
 - **In scope:**
 - IOC application: database, SNL sequencer program, C helper library, autosave configuration, startup scripts
 - a PV-level simulator of the Alicat and the O2 analyzer for the bench
 - an automated acceptance test suite
 - Phoebus screens (§13)
 - the production start-up entry
 - **Out of scope:**
 - any change to the Alicat IOC (`15Alicat`) or its database
 - any change to existing PVs, screens or Phoebus configuration
 - hardware changes
 - the cylinder-pressure PV (it does not exist yet; §6.3)
-

2. Hard rules (MUST)

2.1 Beamline safety

1. **No writes to `15IDC:* PVs`** by any tool (caput, pyepics, IOC links) until the user explicitly lifts this rule. As of 2026-09-25 an experiment is running on them. **Reads** are allowed only when you actually need them.
2. **Bench IOCs and simulators must be confined to localhost.** Source `~/epics-sim-env.sh` in MSYS2, or set the same variables:
 - `EPICS_CA_ADDR_LIST=127.0.0.1`, `EPICS_CA_AUTO_ADDR_LIST=NO`
 - `EPICS_CAS_INTF_ADDR_LIST=127.0.0.1`, `EPICS_CAS_BEACON_ADDR_LIST=127.0.0.1`,
`EPICS_CAS_AUTO_BEACON_ADDR_LIST=NO`
 - the PVA equivalents
 - Note: `~/bash_profile` sets `EPICS_CA_ADDR_LIST` to a beamline host.
3. **Bench PV prefixes** must never start with `15ID`:
 - controller: `SIM:SampleGas:` (station 1), `SIM2:SampleGas:` (station 2)
 - simulated Alicat: `SIM:Alicat1:`
 - simulated O2: `SIM:O2`
4. **Only these existing PVs may ever be written**, and only by the production IOC once writes are enabled (§8.20): `15IDC:Alicat1:Setpoint`, `15IDC:Alicat1:RampRate`, `15IDC:Alicat1:Run`. The user owns the Alicat IOC and approved exactly these.
5. **Never reset the Alicat totalizer** (`ResetTotal`, `SetTotal`). The ledger only reads `Total_RBV`.
6. **Hardware commissioning** (§15.3) happens only with the user present.

2.2 This Windows machine

- **Python:** `%USERPROFILE%\venvs\bluesky\Scripts\python.exe` (3.12; has caproto, pyepics, numpy, matplotlib). `python` on PATH is a broken Microsoft Store stub. Do not install into the Anaconda base environment.
- **Shell:** PowerShell 5.1 (no `&&`). Git Bash is also available. For MSYS2, run `C:\msys64\usr\bin\bash.exe -l script.sh` with `MSYSTEM=MINGW64` and `CHERE_INVOKING=1`. Put commands in script files: PowerShell quoting mangles `-c` strings.

- **Never write code through shell heredocs;** use a file-writing tool (heredocs have corrupted escapes here before).

2.3 EPICS bench on this machine (already built; verified 2026-09-25)

- **Base:** 7.0.8.1 MinGW at C:\msys64\home\base-7.0.8.1 (MSYS path /home/base-7.0.8.1).
- **Modules** in /home/support: seq R2-2-9, sscan R2-12, calc R3-8, autosave R6-0, asyn R4-46, std R3-6-4. Shared /home/support/RELEASE.local.
- **Working example:** ~/bench/smoke links epid with an SNL program and runs.
- **Traps, each already hit:**
 1. gcc 16 defaults to C23. base/configure/CONFIG_SITE.local sets USR_CFLAGS += -std=gnu17. Keep it.
 2. std requires asyn.
 3. **SNL code must be in a LIBRARY_IOC (DLL)**, not in PROD_IOC sources. Otherwise the link fails with undefined reference to __imp_pvar_func<prog>Registrar.
 4. **The generated envPaths contains MSYS paths** (/home/...). The native exe needs C:/msys64/home/..., so rewrite it before running.
 5. **The IOC exe needs the support bin/windows-x64-mingw directories on PATH** for the DLLs.
- **Version gap:** production versions differ (§11.2). Write code compatible with **both** (calc R3-7-5 / R3-8, asyn R4-44-2 / R4-46).

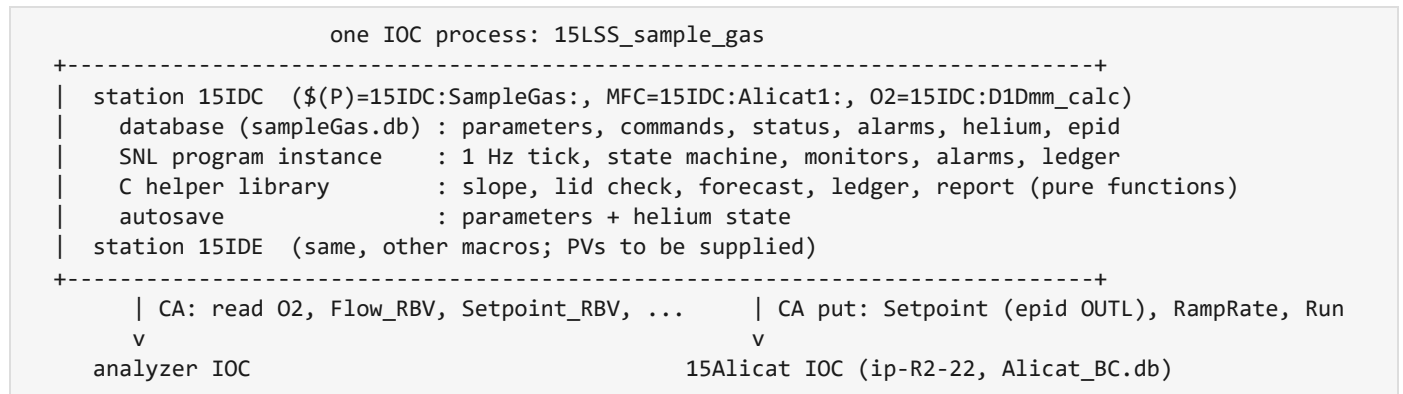
2.4 Repository

- **Location:** work in the o2-purge repo (outside OneDrive on purpose).
- **Commit identity** (already set repo-locally; verify before the first commit): Dave-J-W <248028152+Dave-J-W@users.noreply.github.com>.
- **Do not break the simulator.** If you touch simulator/, its self-test must still pass 19/19.

3. Physical context (summary)

- **Enclosure:** ~41 L, purged with helium through the Alicat. The O2 analyzer (15IDC:D1Dmm_calc) reads 1 Hz. Ambient reads 19.0–19.6 %.
 - **Normal lid (mode A):** holds ~0.99 % at ~0.25 SLPM. The box time constant there is ~2.7 h.
 - **Collimator lid (mode B):** needs ~0.84 SLPM, and seals better at higher flow.
 - **Dead time at hold flow ~80 s:** transport ~10–75 s depending on flow, plus a ~65 s sensor-zone lag.
 - **At 20 SLPM purge:** O2 decays as $\exp(-F \cdot t/V)$ with $\tau \approx 2$ min, and the reading lags ~12 s.
 - **Reading noise:** white 0.84 m% per 1 Hz sample, plus a slow component (σ 0.67 m%, τ 79 s) that is **not** bulk O2.
 - **Surface limit:** flow above ~2 SLPM may vibrate the liquid surface under study.
 - **Alicat quirks:**
 - the setpoint is ignored while on hold
 - ramp rate normally 3 SLPM/s
 - resolution 0.01 SLPM
 - Total_RBV totalizer
 - **Full detail:** design spec §2 and §2.1.
-

4. Architecture



- **One IOC, N stations.** Each station is one `dbLoadRecords` of the station template and one `seq` instance with its own macros (§5.4). There are no shared variables between stations; the SNL program is compiled reentrant (+r).
- **Division of labour:**
 - `epid` does the PID arithmetic, with its own anti-windup and bumpless enable. Its OUTL writes the Alicat setpoint.
 - **The SNL program** is the controller: tick scheduling, input validation, state machine, commands, lid detection, hold monitor, epid configuration and triggering, alarms, settling logic, cylinder forecast, helium ledger, logging.
 - **The C helper library** holds the numerical algorithms, as pure functions with unit tests.
- **The epid is triggered by the SNL** (`SCAN Passive`, SNL writes `.PROC` every `pidScan` seconds after the tick), so ordering is deterministic, matching the reference.
- **The O2 mean fed to epid** is written by the SNL into a soft `ai` that epid reads through INP. This reproduces the reference's "mean of the last `avgN` valid samples".

5. IOC application layout and startup

5.1 Names

Item	Value
IOC name (<code>procServ</code> , <code>start_ioc</code>)	<code>15LSS_sample_gas</code>
Top directory	<code>lssSampleGas</code> (production: under <code>support/ChemMat/</code> ; see §11)
App directory	<code>lssSampleGasApp</code>
IOC binary and dbd	<code>lssSampleGas</code> . It must start with a letter: EPICS uses it as a C identifier.
<code>iocBoot</code> directory	<code>iocBoot/iocLSS_sample_gas</code>
SNL program name	<code>sampleGas</code>
<code>procServ</code> port (production)	20125

5.2 Modules

- **Required:** `base 7.0.x`, `std` (`epid`), `calc`, `seq`, `autosave`, `asyn` (dependency of `std`).
- **dbd:** `base.dbd`, `asyn.dbd`, `stdSupport.dbd`, `calcSupport.dbd`, `asSupport.dbd`, and the SNL registrar dbd.

- **Libraries:** `sampleGasSupport` (the DLL with the SNL code and C helpers), `std`, `calc`, `sscan`, `autosave`, `asyn`, `seq`, `pv`, plus base IOC libs.

5.3 Database files

File	Content
<code>sampleGas.db</code>	Station template: everything in §7, instantiated once per station
<code>sampleGasModes.db</code>	Four mode slots (A–D), included or loaded per station (§7.4)
<code>sampleGas_settings.req</code>	Autosave: parameters and mode slots (§10)
<code>sampleGas_helium.req</code>	Autosave: helium state and arrays (§10)

5.4 Macros per station

Macro	Meaning	15IDC production	Bench
P	Controller prefix	15IDC:SampleGas:	SIM:SampleGas:
MFC	Alicat prefix (\$(P)\$ (R) of Alicat_BC.db)	15IDC:Alicat1:	SIM:Alicat1:
O2	O2 reading PV (full name)	15IDC:D1Dmm_calc	SIM:O2
CYL	Cylinder-pressure PV, or empty	(empty)	(empty)
STN	Station label for messages	15IDC	SIM

15IDE: the macros are to be supplied by the user (open item §16).

5.5 `st.cmd` (sketch; exact form is the implementer's)

```
< envPaths
dbLoadDatabase("dbd/lssSampleGas.dbd")
lssSampleGas_registerRecordDeviceDriver(pdbbase)
< save_restore.cmd # autosave paths, pass0/pass1 restore of both .req fil
dbLoadRecords("db/sampleGas.db", "P=15IDC:SampleGas:,MFC=15IDC:Alicat1:,O2=15IDC:D1Dmm_calc,CYL=,STN=15I
# dbLoadRecords("db/sampleGas.db", "P=15IDE:SampleGas:,MFC=<tbd>,O2=<tbd>,CYL=,STN=15IDE")
iocInit
create_monitor_set("sampleGas_settings.req", 30, "P=15IDC:SampleGas:")
create_monitor_set("sampleGas_helium.req", 300, "P=15IDC:SampleGas:")
seq sampleGas, "P=15IDC:SampleGas:,MFC=15IDC:Alicat1:,O2=15IDC:D1Dmm_calc,CYL=,STN=15IDC"
```

- **Restore order:** parameters must be restored **before** `iocInit` (pass 0). The SNL program must not act until parameters are valid; see the IOC-start rules in §8.15.

6. External interface (PVs owned by other IOCs)

6.1 Alicat (\$(MFC) = 15IDC:Alicat1:)

Behaviour verified from `ipApp/Db/Alicat_BC.db` and `.proto`. The deployed copy is identical to `epics-modules/ip master`.

PV	Type	Controller use	Notes
\$(MFC)Setpoint	ao, stream	Write (epid OUTL, and SNL puts)	SDIS=\$(MFC)Running_RBV, DISV=0 : while the Alicat is on hold, a put changes VAL but the record does not process, so nothing reaches the device. PREC 3. Engineering units SLPM.
\$(MFC)Setpoint_RBV	ai, soft	Read (monitor)	Device setpoint, from the 1 Hz poll
\$(MFC)Flow_RBV	ai, soft	Read (monitor)	Measured flow, SLPM, 1 Hz; quantised 0.01
\$(MFC)Total_RBV	ai, soft	Read (monitor)	Totalizer, standard litres. Never reset.
\$(MFC)Running_RBV	bi	Read (monitor)	1 = running, 0 = paused (status contains HLD or EXH)
\$(MFC)Status	stringin	Read (optional, for logging)	Status letters, e.g. HLD
\$(MFC)RampRate	ao, stream	Write (clamps only, §8.4 and §8.6)	Sends SR <v> 4: SLPM per second (verified on the wire). 0 = no ramp (instant).
\$(MFC)RampRate_RBV	ai	Read	Processed in the poll chain
\$(MFC)Run	bo, stream	Write 1 (hold monitor, precheck)	Sends C: cancel hold. The device resumes its own stored setpoint.
\$(MFC)Gas_RBV	mbbi	Read	Index 7 = He
\$(MFC)FlowUnits_RBV	stringin	Read at start	Expect SLPM

Link and monitor rules:

- Reads use CA monitors: in SNL, assign + monitor.
- Every read must carry **connection state and severity** into the logic. A disconnected or INVALID O2 counts as "O2 invalid" (§8.2).
- A disconnected Alicat is handled in §8.18.

6.2 O2 analyzer (\$(O2) = 15IDC:D1Dmm_calc)

- A calc record (DMM channel), % O2, updating at ~1 Hz.
- Monitor it and take the **latest value and severity at each tick**.
- Severity INVALID, or no connection, means invalid.
- The value may repeat if the source stalls; this is the "frozen" rule, §8.2.

6.3 Cylinder pressure (\$(CYL), optional)

- It does not exist yet.
- If CYL is empty or disconnected, publish "n/a" and **do not use it in any logic**. The litres and forecast come from Total_RBV (§8.16).

7. Controller PV interface (per station)

Conventions:

- **Parameters** (Par:, Mode:) are ao/longout/stringout/mbbo with DRVL/DRVH set to the limits in §9, are autosaved, and are monitored by the SNL.
- **Commands** (Cmd:) are bo records that the SNL resets to 0 after acting.

- **Status** (Sts:) and diagnostics (Diag:) are written only by the SNL.
- **Alarms** (Alm:) are mbbi, value 0/1/2 = none/MINOR/MAJOR, with ZRSV=NO_ALARM, ONSV=MINOR, TWSV=MAJOR, so the alarm server and archiver see real EPICS severities. Each has a companion Alm:<Name>:Msg (lsi, 256 chars).
- **Long texts** use lsi/lso (base 7).
- **Times** are POSIX epoch seconds (double) unless stated otherwise.

7.1 Operator (main panel)

PV	Type	Meaning
\$(P)Mode	mbbo (4 states)	Enclosure mode A/B/C/D. The state strings are copied from Mode:X:name at start and on change. Autosaved.
\$(P)Par:target	ao	O2 target, %. Limits 0.2–5. Autosaved. A change while REGULATE starts settling (§8.12).
\$(P)Cmd:Purge	bo	Start a purge (§8.5)
\$(P)Cmd:FlowZero	bo	Go to FLOW_ZERO
\$(P)Cmd:ResumePID	bo	OPEN_LOOP → HANDOFF, if O2 is valid
\$(P)Cmd:NewCylinder	bo	New helium cylinder fitted (§8.16)
\$(P)Sts:State	mbbi	0 IDLE, 1 PRECHECK, 2 PURGE, 3 HANDOFF, 4 REGULATE, 5 OPEN_LOOP, 6 FLOW_ZERO, 7 OPEN_STOP
\$(P)Sts:StateDesc	lsi	One-line description of the state (texts from the reference STATES)
\$(P)Sts:LastAction	lsi	Most recent transition reason or override text
\$(P)Sts:Progress	lsi	State detail, as the simulator's progress line: purge timer and lid check; handoff phase; settling and PID internals
\$(P)Sts:InRange	bi	1 if the O2 is valid and $ O2 - target \leq tol$
\$(P)Sts:O2	ai	Copy of the O2 reading used this tick (NaN or INVALID severity when invalid)
\$(P)Sts:O2Valid	bi	The o2ok flag of §8.2
\$(P)Sts:ExpectedFlow	ai	SLPM (§8.13)
\$(P)Sts:LastCmd	ai	Last flow commanded, SLPM (NaN before the first command)
\$(P)Sts:Heartbeat	longin	+1 every tick
\$(P)Sts:Banner	lsi	Active alarm texts, most severe first, "; "-separated, or No alarms
\$(P)Sts:WorstSevr	mbbi	Worst active alarm, 0/1/2 (with severities)
\$(P)Sts:WriteEnable	bi	Mirror of Par:writeEnable (§8.20)
\$(P)He:LeftL	ai	Litres left in the cylinder
\$(P)He:EmptyMinH / EmptyMaxH / EmptyMedianH	ai	Run-out forecast across windows, hours (NaN if none)
\$(P)He:ForecastText	lsi	E.g. empty in 5.5 d–5.6 d (median 5.5 d, 5 windows) or run-out forecast: collecting data (needs ≥ 6 h)
\$(P)Sts:CylPressure	ai	Copy of \$(CYL) if configured; otherwise INVALID with text n/a

7.2 Admin and deep-admin parameters

- **Every parameter in §9** exists as `$(P)Par:<key>`, using the exact key names given there (e.g. `$(P)Par:purgeTimeoutMargin`).
- **Limits and defaults:** `DRVL/DRVH` = the §9 limits. `VAL` = the §9 default, set in the database so a fresh IOC with no autosave file behaves like the reference.
- **Integer parameters** are `longout`.
- `$(P)Par:writeEnable` (bo) is new, not in the simulator (§8.20).

7.3 Admin commands

PV	Meaning
<code>\$(P)Cmd:ReleaseIdle</code>	Go to IDLE ("admin released control")
<code>\$(P)Cmd:MarkNewRun</code>	Ledger event <code>newrun</code> (§8.17)
<code>\$(P)Cmd:ResetOverrideCount</code>	Set <code>Diag:OverrideCount</code> = 0

7.4 Mode slots (A, B, C, D)

- **PVs:** `$(P)Mode:<X>:name` (stringout), `:baseFlow`, `:n`, `:KP`, `:KI`, `:drvh`, `:drv1` (ao, limits in §9.2). All are autosaved.
- **Defaults:**

Slot	name	baseFlow	n	KP	KI	drvh	drv1
A	Normal lid	0.25	1.0	−9.3	8.8e-4	1.0	0.05
B	Collimator lid	0.84	0.53	−10	1.4e-3	2.0	0.3
C	Spare C	0.25	1.0	−9.3	8.8e-4	1.0	0.05
D	Spare D	0.25	1.0	−9.3	8.8e-4	1.0	0.05

- Slot C is intended for the planned tighter enclosure. KP values are stated "at 0.99 %" (gain scheduling, §8.11).

7.5 Alarms (Alm:)

Name	Levels used	Key in reference
<code>Alm:OpenStop</code>	2	<code>openStop</code>
<code>Alm:PurgeIncomplete</code>	1 (latched 300 s)	<code>purgeInc</code>
<code>Alm:O2Bad</code>	2	<code>o2bad</code>
<code>Alm:OpenLoop</code>	2	<code>openLoop</code>
<code>Alm:Override</code>	1 (latched 300 s)	<code>override</code>
<code>Alm:HoldStuck</code>	2	<code>holdStuck</code>
<code>Alm:Mismatch</code>	2	<code>mismatch</code>
<code>Alm:FlowHigh</code>	1, 2	<code>flowHigh</code>
<code>Alm:FlowLow</code>	1	<code>flowLow</code>
<code>Alm:Pinned</code>	2	<code>pinned</code>
<code>Alm:O2High</code>	1, 2	<code>o2High</code>

Name	Levels used	Key in reference
Alm:NotReached	1	notReached
Alm:CylLow	1, 2	cylLow
Alm:Gas	1	gas

Conditions, texts and clearing rules: §8.14. A heartbeat or IOC-down alarm cannot come from the IOC itself; it is the client's job (§13.4).

7.6 Diagnostics (Diag: , read-only)

- **Lid detector:** AboveCount (s), MaxRate (%/s), LidArmed
- **Hold monitor:** HoldSec, HoldAttempts
- **Settling:** Settling (bi), SettleDir (−1/0/+1), SettleT0, TowardRate (%/min), O2Slope (%/min), StallSec, StallLatched
- **Alarm gating and PID:** FlowSteady, InFine, PinnedSec
- **Purge:** PurgeElapsed, PurgeTimeout, O2Start, LidOnsetT, LidRatio, LidCurv, LidResult (mbbi: pending / passed / skipped / open), EstO2 (lag-corrected), Blind, HandoffPhase
- **Overrides:** OverrideCount, and OverrideLog (lsi, last 20 overrides, newest first, one per line, hh:mm:ss text)
- **Log:** Log:Text (waveform CHAR, 16384): the event log, newest first, format YYYY-MM-DD hh:mm:ss [MINOR|MAJOR] message. Keep the last ~200 lines.

The epid record (\$(P)PID) fields are directly readable.

7.7 Helium ledger and usage (He:)

PV	Type	Meaning
He:CumL	ao (autosaved)	Monotonic litres dispensed (§8.17)
He:LastTotal	ao (autosaved)	Last Total_RBV seen
He:CylBase	ao (autosaved)	Total_RBV at the last "new cylinder"
He:HistT, He:HistUsed	waveform DOUBLE, NELM 6000 (autosaved) + He:HistN	1-min usage log, 4 days
He:EvT, He:EvType, He:EvL	waveforms NELM 4000 (autosaved) + He:EvN	Ledger events. Types: 1 purge, 2 zero, 3 cylinder, 4 newrun
He:SnapT, He:SnapL	waveforms NELM 2000 (autosaved) + He:SnapN	Hourly snapshots of CumL
He:Est<W>H for W ∈ {3d, 2d, 1d, 12h, 6h}	ai	Run-out per window, hours (NaN if none)
He:Rate<W>	ai	Usage rate per window, L/day
He:Rep:Dispensed, :InRuns, :Cylinders, :CylEquiv, :WinStart, :WinEnd	ai	Usage report (§8.17)
He:Rep:RunStart, :RunEnd, :RunPurges, :RunL, :RunCyl, :RunFinished	waveforms NELM 100 + He:Rep:RunN	Runs in the report window. RunEnd = 0 while open.
He:Rep:Text	waveform CHAR 8192	Human-readable report, as the reference's Admin text

7.8 epid and averaging records

PV	Type	Settings
\$(P)PID	epid	SCAN Passive (processed by SNL). INP=\$(P)PID:CVAL NPP. OUTL=\$(MFC)Setpoint PP. KD=0. FMOD=PID. FBON written by SNL only. VAL, KP, KI, DRVL, DRVH, ODEL written by SNL each PID step (§8.11). PREC=4. EGU=SLPM.
\$(P)PID:CVAL	ai (soft)	Mean of the last avgN valid O2 samples, written by SNL before each PID step

8. Behaviour (normative)

Pseudocode names follow the reference (Controller methods). "Log" means: append to Log:Text with a timestamp and severity, and print to the IOC console (errlogPrintf), which appears in the procServ log.

8.1 Timing model

- **One tick per second**, aligned to wall-clock seconds. Use epicsTime and compensate for drift; do not accumulate delay(1.0). Let now be the tick time in integer epoch seconds.
- **Inputs are sampled once per tick**, at the start. That tick then works on one consistent snapshot: O2 value and severity, Flow_RBV, Setpoint_RBV, Running_RBV, RampRate_RBV, Gas_RBV, Total_RBV, connection states.
- **Order of operations in every tick** (the reference tick()):
 1. **Read inputs** (§8.2).
 2. **Expire latched alarms** whose latch time has passed.
 3. **If state ≠ IDLE**: run the **hold monitor** (§8.7).
 4. **Run the lid detector** (§8.6). If it trips, skip step 5.
 5. **State handler** for the current state (§8.4).
 6. **Ledger update** (§8.17).
 7. **Cylinder forecast** (§8.16).
 8. **Alarm checks** (§8.14).
 9. **If now mod pidScan == 0**: run the **PID step** (§8.11).
 10. **Publish** all Sts:, Diag:, Alm: and He: PVs that changed; increment Sts:Heartbeat.
- **Operator commands** (Cmd:*) are handled at the start of the next tick, after step 1, in this order: FlowZero, Purge, ResumePID, ReleaseIdle, NewCylinder, MarkNewRun, ResetOverrideCount. Each is reset to 0 after handling. The reference acts on button presses immediately; acting at the next tick is accepted.
- **Parameter changes** take effect at the next use. A target change also triggers §8.12.

8.2 Inputs and O2 validity (readInputs)

1. o2 = O2 value; sevr = O2 severity. A disconnected PV counts as sevr = INVALID.
2. **Frozen detection**: if sevr < INVALID and o2 equals the previous tick's value exactly, increment sameCount; otherwise set it to 0. frozen = sameCount ≥ frozenTime.
3. o2ok = sevr < INVALID **and** o2Min ≤ o2 ≤ o2Max **and** not frozen.
4. **O2 history**: append o2 to o2hist, keeping at least max(90, stallWindow + slopeAvgN + 5) entries. **This length matters**: a shorter buffer silently disables the slope check.
5. **Rate**: if the history has more than 10 entries, rate = (o2 - o2hist[n-11]) / 10 (%/s); else 0. Append (o2ok ? rate : 0) to rateHist, keeping lidSlopeWindow entries. maxRate = max(0, max(rateHist)).
6. **Level**: aboveCount = (o2ok and o2 > lidLevel) ? aboveCount + 1 : 0.
7. **Average**: if o2ok, append o2 to avgBuf, keeping the last avgN.

8. **Alarm:** if not `o2ok`, raise `Alm:O2Bad = 2` with text `O2 reading frozen` (if frozen) or `O2 reading invalid`. Otherwise clear it.

8.3 Commanding the flow (command)

- `command(v)`: $v = \max(0, v)$. If v differs from `lastCmd` by more than $1e-9$, set `lastCmd = v` and record `lastCmdTime` and the current flow. Then put `$(MFC)Setpoint = v`.
- **Only when writes are enabled** (§8.20).
- **While FBON = 1, epid writes the setpoint** through OUTL, and the SNL does not write it (except for the hold-monitor re-send, §8.7). After each PID step the SNL copies `PID.OVAL` into `lastCmd`.

8.4 States

Common entry actions (`enter(s, reason)`), performed on every transition:

- Set the state.
- Reset the per-state data (timer `t0 = now`).
- If $s \neq \text{REGULATE}$: `PID.FBON = 0`.
- Clear, silently, alarms `FlowHigh`, `FlowLow`, `Pinned`, `O2High`, `OpenLoop`, `NotReached`, and reset their debounce state. Reset `pinnedSec` and `pinnedLowSec`. `settling = false`.
- If $s = \text{REGULATE}$: `startSettling("feedback on")` (§8.12).
- `LastAction = "<STATE>: <reason>". Log "<prev> → <STATE> (<reason>)"`.

Then the state-specific entry actions:

State	Entry actions
IDLE	none
PRECHECK	Clear <code>OpenStop</code> and <code>PurgeIncomplete</code> silently
PURGE	Ledger event <code>purge</code> . Initialise purge data: <code>o2Start = o2</code> , <code>timerT0 = none</code> , <code>dropChecked = false</code> , <code>below = 0</code> , <code>onsetT = none</code> , <code>cOn = none</code> , <code>kin = none</code> . <code>lidArmed = false</code> . <code>command(purgeFlow)</code> .
HANDOFF	If $0 < \text{RampRate_RBV} < \text{rampMinHandoff}$: put <code>RampRate = rampMinHandoff</code> , then <code>override("ramp rate <r> → <min> SLPM/s (handoff minimum)")</code> . <code>Phase = settle</code> . <code>command(expectedFlow)</code> .
REGULATE	(common actions above). The caller then runs <code>configEpid</code> and sets <code>FBON = 1</code> .
OPEN_LOOP	<code>command(expectedFlow)</code> . Raise <code>Alm:OpenLoop = 2</code> , <code>O2 unavailable: running blind at fixed flow</code> .
FLOW_ZERO	Ledger event <code>zero</code> . <code>command(0)</code> .
OPEN_STOP	Ledger event <code>zero</code> . <code>command(0)</code> . (The caller also raises <code>Alm:OpenStop</code> , §8.6 and §8.9.)

Per-tick handlers (step 5):

State	Handler
IDLE	nothing (the controller does not own the flow)
PRECHECK	§8.8, runs once, then enters PURGE
PURGE	§8.9 and §8.10
HANDOFF	If not <code>o2ok</code> : enter <code>OPEN_LOOP</code> (<code>O2 unavailable during handoff</code>). Else <code>command(expectedFlow)</code> . Phase settle: if $ \text{Flow_RBV} - \text{expected} \leq \text{handoffFlowTo1}$, or more than 30 s since entry, go to phase delay ($t1 = \text{now}$). Phase delay: once $\text{now} - t1 \geq \text{fbDelay}$, enter <code>REGULATE</code> (<code>feedback on</code>), <code>configEpid</code> , <code>FBON = 1</code> .
REGULATE	If not <code>o2ok</code> : enter <code>OPEN_LOOP</code> with reason <code>O2 reading frozen</code> or <code>O2 reading invalid</code> .

State	Handler
OPEN_LOOP	command(expectedFlow) (tracks target and mode changes)
FLOW_ZERO, OPEN_STOP	nothing (flow stays 0)

8.5 Operator commands

Command	Allowed when	Action
Purge	state $\notin$ {PRECHECK, PURGE}	enter PRECHECK (operator pressed Purge)
FlowZero	always	enter FLOW_ZERO (operator pressed Flow Zero)
ResumePID	state = OPEN_LOOP and o2ok	enter HANDOFF (operator pressed Resume PID)
ReleaseIdle	always	enter IDLE (admin released control)
NewCylinder	always	\$8.16
MarkNewRun	always	ledger event newrun; log admin: start of a new user run marked

- **Target change:** log operator: 02 target $\rightarrow$ <v> %. If REGULATE, startSettling("target change").
- **Mode change:** log operator: enclosure mode $\rightarrow$ <name>. No other immediate action.
- **Rejected commands** (not allowed): reset the command to 0 and log <command> ignored in <STATE>. (The reference ignores them silently; logging is an accepted addition.)

8.6 Lid detector (lidDetector), every tick before the state handler

- **Arming during a purge:** if state = PURGE and o2ok and o2 < lidArmLevel, then lidArmed = true.
- **Armed** = state $\in$ {HANDOFF, REGULATE}, or (state = PURGE and lidArmed).
- **Trip:** if armed and o2ok and aboveCount $\geq$ lidFilter and maxRate $\geq$ lidSlope, enter OPEN_STOP and raise Alm:OpenStop = 2. Text: enclosure opened, flow stopped (02 <o2:.1f> %, rise <maxRate:.2f> %/s). Skip the state handler this tick.
- **What trips it:** a lid lift trips ~10 s after onset. A slow creep, e.g. hours with the flow off or a cracked lid, never does, because its rate stays far below lidSlope.

8.7 Hold monitor (holdMonitor), every tick when state $\neq$ IDLE

- **If Running_RBV = 0:**
 - increment holdSec
 - if holdAttempts $\geq$ holdRetries: raise Alm:HoldStuck = 2, MFC on hold, cannot resume
 - if holdSec $\geq$ holdDetect and now $\geq$ nextHoldAttempt:
 - put \$(MFC)Run = 1
 - holdAttempts ++, holdRecovering = true
 - log MINOR MFC on hold: wrote Run (attempt <n>)
 - nextHoldAttempt = now + (holdAttempts < holdRetries ? holdRetryInterval : holdSlowRetry)
- **Else (running):**
 - if holdRecovering:
 - holdRecovering = false
 - if lastCmd is known, put \$(MFC)Setpoint = lastCmd **with processing**. Writes made during the hold never reached the device.
 - clear Alm:HoldStuck
 - override("MFC was on hold, resumed; setpoint <lastCmd:.2f> re-sent")
 - reset holdSec, holdAttempts and nextHoldAttempt to 0

- **Why IDLE is excluded:** the controller does not own the flow in IDLE. The hold monitor runs in FLOW_ZERO and OPEN_STOP because a held valve can keep helium flowing even at a zero setpoint.

8.8 PRECHECK (doPrecheck): correct and warn, never refuse

1. If Running_RBV = 0: put Run = 1, holdRecovering = true, log MINOR PRECHECK: MFC on hold, wrote Run.
2. If RampRate_RBV > rampMaxPurge or RampRate_RBV = 0 (0 = instant): put RampRate = rampMaxPurge, then override("ramp rate <r or '0 (= no ramp, instant)'> → <max> SLPM/s (purge maximum)").
3. If Gas_RBV ≠ He: raise Alm:Gas = 1, MFC gas table is <gas>, not He (flow reading wrong). Otherwise clear it.
4. blind = not o2ok.
5. Enter PURGE, with reason blind purge: O2 unavailable, timer only or purge started.

Optional: if FlowUnits_RBV ≠ SLPM, raise MAJOR MFC flow units are <u>, not SLPM. This is in the design spec §4.1 but not in the reference; implement it as an alarm only.

8.9 PURGE: timer, blind mode and lid check (doPurge, first part)

1. **Purge timer:** if timerT0 is unset and (Flow_RBV ≥ 0.95·purgeFlow, or 30 s since entry), set timerT0 = now. e1 = now - timerT0 (0 while unset). Publish Diag:PurgeElapsed.
2. **O2 lost during the purge:** if not o2ok and not blind: blind = true, and log MAJOR O2 lost during purge: continuing on the timer (blind).
3. **Timeout** = purgeTimeout() (§8.10). Publish it.
4. **Blind mode:** if blind, then when e1 ≥ timeout, enter OPEN_LOOP (blind purge complete). Return; do nothing else in blind mode.
5. **Lid check** (only if dropChecked is false and timerT0 is set):
 - **Skip:** if o2Start < dropSkipLevel (or o2Start is not a number): dropChecked = true, result = skipped, and log lid check skipped: purge started at <o2Start:.2f> % (< <skip> %).
 - **Otherwise:**
 1. **Onset:** if onsetT is unset and o2 ≤ o2Start·(1 - lidOnsetFrac), then onsetT = e1 and cOn = o2.
 2. **No onset:** if onsetT is unset and e1 ≥ lidOnsetMax: dropChecked = true, result = open. Enter OPEN_STOP and raise Alm:OpenStop = 2, no O2 decay within <lidOnsetMax> s of full flow: enclosure open?. Return.
 3. **Decision,** once onsetT is set and e1 ≥ onsetT + lidWindow:
 - dropChecked = true
 - kExp = purgeFlow / V / 60 (1/s); span = e1 - onsetT; half = floor(span / 2)
 - cMid = the O2 sample span - half ticks before now (from o2hist)
 - kObs = ln(cOn / o2) / span; k1 = ln(cOn / cMid) / half; k2 = ln(cMid / o2) / (span - half)
 - ratio = kObs / kExp; curv = k1 > 0 ? k2 / k1 : 0
 - store ratio, curv, kObs, checkAt = e1, cCheck = o2
 - **Open** if ratio < openSlopeFrac or curv < lidCurvMin: enter OPEN_STOP and raise Alm:OpenStop = 2, purge decay <100·ratio:.0f> % of the lid-on rate, curvature <curv:.2f>: enclosure open?. Return.
 - **Passed:** log lid check passed at <e1> s: decay <...> % of the lid-on rate (open < <...> %), curvature <...> (open < <lidCurvMin>).

8.10 PURGE: handoff and timeout (doPurge, second part; purgeTimeout)

1. **Lag-corrected O2:** k = (kObs > 0 ? kObs : purgeFlow/V/60); est = o2 · exp(-k · purgeLag). Publish Diag:EstO2.

2. If `est < target - delta`: `below++`; else `below = 0`.
3. **Handoff**: if `below ≥ handoffHold`, enter HANDOFF with reason `lag-corrected O2 <est:.3f> % < target - Δ` for `<handoffHold> s` (reading `<o2:.3f> %`). Return.
4. **Timeout**: if `e1 ≥ timeout`, latch `Alm:PurgeIncomplete = 1` for 300 s with text `purge incomplete: timeout (<h:mm:ss>)` reached before `target - Δ`, then enter HANDOFF (purge timeout).
5. **purgeTimeout()**:
 - `goal = max(0.05, target - delta)`; `extra = purgeLag + handoffHold`
 - **After a lid check with kObs > 0**: `t = checkAt + purgeTimeoutMargin · (ln(max(cCheck, goal)/goal)/kObs + extra)`
 - **Otherwise**: `c0 = (blind or o2Start ≤ 0) ? ambientRef : o2Start`; `k = purgeFlow/V/60`; `t = purgeTimeoutMargin · (ln(max(c0, goal)/goal)/k + extra)`
 - Clamp `t` to `[purgeTimeoutMin, purgeTimeoutMax]`. It is recomputed every tick.

8.11 PID step (configEpid and processEpid), when `now mod pidScan == 0`

1. **Skip** if `avgBuf` is empty.
2. **Configure epid**:
 - `VAL = target`
 - `KP = gainSchedule ? mode.KP · (0.99/target) : mode.KP`
 - `KI = mode.KI`
 - `ODEL = odel`
 - **Fine band**, with hysteresis on the **previous** CVAL:
 - if `fineBand > 0` and CVAL is known: `err = |VAL - CVAL|`
 - if `inFine` and `err > 2 · fineBand`: `inFine = false`; else if not `inFine` and `err < fineBand`: `inFine = true`
 - if `inFine`: `KP ×= fineKPx`, `KI ×= fineKIx`
 - if `fineBand = 0`, or CVAL is unknown: `inFine = false`
 - `DRVH = min(mode.drvh, hardCeiling)`; `DRVL = min(DRVH, mode.drvl)`
3. **Input**: write `PID:CVAL = mean(avgBuf)`.
4. **Process** `$(P)PID` (put `PROC`) and wait for completion (put-callback). `epid` then applies its own algorithm:
 - `error = VAL - CVAL`
 - bumpless start on `FBON 0→1`, initialising `I` from `OUTL`, which is the current `Setpoint VAL`
 - anti-windup against `DRVL/DRVH`
 - `ODEL`
 - writes `OUTL ($MFC)Setpoint` while `FBON = 1`
5. **After processing**: `lastCmd = PID.OVAL` if `FBON = 1`.

Reference equivalence. The reference's `processEpid` reimplements `devEpidSoft`:

- $p = KP \cdot e$; $\Delta I = KP \cdot KI \cdot e \cdot dt$
- `I` integrates only when `OVAL` is strictly inside the limits, or when ΔI moves it back inside
- `I` is clamped to the limits
- `out = clamp(p + I)`
- `OVAL` updates only if $|out - OVAL| > ODEL$

Bench-verify that the real `epid` matches. In particular, check whether `OUTL` is written every processing even when `OVAL` is unchanged. If it is not, nothing breaks: the hold monitor re-sends explicitly.

8.12 Settling (startSettling and the settling part of alarmChecks)

- `startSettling(why)`:

- `stallSec = 0, stallLatched = false`
- if `o2` is known and $|o2 - target| \leq tol$: `settling = false`, and return
- otherwise: `settling = true, settleT0 = now`, clear `NotReached` silently, `settleDir = sign(target - o2)`, and log `settling (<why>): O2 falling|rising toward <target:.3f> %`
- **It is called** on `REGULATE` entry and on a target change while `REGULATE`.
- **Every tick in `REGULATE`** (in `alarmChecks`):
 - **Slope:**
 - `W = stallWindow, A = slopeAvgN, n = history length`
 - if `o2ok` and $n > W + A$: `slope = (mean of the last A samples - mean of the A samples ending W ticks earlier) / W · 60 (%/min)`
 - `towardRate = (o2 < target ? slope : -slope)`
 - `judging = settling and now - settleT0 ≥ stallGrace`
 - if `judging` and `towardRate < progressMin`: `stallSec ++`; else if not `stallLatched`: `stallSec = 0`
 - **Settled:** if `settling` and `o2ok` and `O2` has **reached the target in the settling direction** (`settleDir < 0: o2 ≤ target; > 0: o2 ≥ target; 0: |o2 - target| ≤ tol`): `settling = false, stallLatched = false, stallSec = 0`, clear `NotReached`, and log `settled: O2 reached <target> ± <tol> % after <h:mm:ss>`.
 - **Settle timeout:** if `settling` and `now - settleT0 ≥ settleTimeout`: raise `Alm:NotReached = 1`, target not reached within `<h:mm:ss>`.
 - **Stalled** = `stallSec ≥ stallTime`, or (`settling` and the settle timeout has passed). If `settling` and stalled and not `stallLatched`: `stallLatched = true`, and log `settling stalled: O2 no longer approaching the target, process alarms now active`.
 - **Alarm gate:** `alarmsOn = not settling, or stallLatched`.

8.13 Expected flow

- **expectedFlow** = `mode.baseFlow · (0.99 / target)^mode.n`
- **Used by:** `HANDOFF` and `OPEN_LOOP` flows, the flow-high/low alarms, and `Sts:ExpectedFlow`.

8.14 Alarms (`alarmChecks`, plus the alarm-raising steps above)

Mechanisms:

- **setAlarm(k, sev, text):** raise the alarm, or change its level or text. Log it (with severity) only when something changes.
- **clearAlarm(k):** log `cleared: <text>` unless it is a silent clear.
- **latch(k, sev, text, 300 s):** raise now, and auto-clear after 300 s (the expiry is step 2 of the tick).
- **debounce(k, level, text1, text2, delay):** the candidate level must persist `delay s` (default `alarmDelay`) before it is applied, changed or cleared. Any change of candidate restarts the timer.
- **override(text):** `OverrideCount ++`, append to `OverrideLog`, `LastAction = text`, and latch `Alm:Override = 1` for 300 s with text `override: <text>`.

Flow mismatch (every tick, any state):

- **Track the setpoint:** when `Setpoint_RBV` changes (by more than $1e-9$), record the change time and the flow at that moment.
- **Evaluate only if state ≠ IDLE and `Running_RBV = 1`:**
 - `allowed = (ramp > 0 ? |sp - flowAtChange| / ramp : 0) + mismatchMargin`
 - `tol = max(mismatchAbs, mismatchFrac · sp)`
 - `off = |sp - Flow_RBV| > tol`
 - if `off` and time since the change $\geq allowed$: `Alm:Mismatch = 2`, flow mismatch: cylinder empty or MFC fault?

- if not off: clear it
- **Otherwise:** clear it, silently.

In **REGULATE only**, after the settling logic (§8.12):

Alarm	Candidate level	Persistence	Text (level 1 / level 2)
FlowHigh	0 unless (alarmsOn and flowSteady). Then $r = \text{Flow_RBV}/\text{expectedFlow}$: $r \geq \text{flowMajorX} \rightarrow 2$; $r \geq \text{flowMinorX} \rightarrow 1$	flowAlarmDelay	flow $\geq$ <minorX>× expected: check enclosure / flow $\geq$ <majorX>× expected: check enclosure seal
FlowLow	1 if alarmsOn and flowSteady and $\text{OVAL}/\text{expectedFlow} \leq \text{flowLowX}$. Judged on PID demand, not measured flow , so an empty cylinder does not read as "wrong mode".	flowAlarmDelay	flow $\leq$ <lowX>× expected: wrong enclosure mode selected?
Pinned	pinnedSec = consecutive ticks with $\text{OVAL} \geq \text{DRVH} - 1\text{e-6}$. Level 2 if alarmsOn and pinnedSec $\geq$ pinnedTime; else cleared.	(inherent)	PID pinned at max flow: check enclosure seal
O2High	Only evaluated while o2ok: 0 unless alarmsOn; $\text{o2} > \text{target} + \text{o2AbnormalOffset} \rightarrow 2$; $\text{o2} > \text{target} + \text{tol} \rightarrow 1$	flowAlarmDelay	O2 above target range / O2 abnormally high

- **flowSteady** = the OVAL history (stallWindow + 1 samples, one per tick) shows $|\text{OVAL} - \text{OVAL } W \text{ ticks ago}| \leq \text{flowSteadyBand}$, **and** $|\text{O2Slope}| < \text{o2SteadyRate}$.
- **Minimum-flow note, no alarm:** count ticks with $\text{OVAL} \leq \text{DRVH} + 1\text{e-6}$ and $\text{o2} < \text{target} - \text{tol}$. When the count reaches exactly pinnedTime with alarmsOn, log note: PID at minimum flow and O2 still below target. The user decided low O2 is not an operator alarm.

Raised elsewhere: OpenStop (§8.6, §8.9), PurgeIncomplete (§8.10), O2Bad (§8.2), OpenLoop (§8.4), HoldStuck (§8.7), Override (§8.14), CylLow (§8.16), Gas (§8.8).

Clearing on transitions: see the common entry actions in §8.4. OpenStop and PurgeIncomplete clear on PRECHECK entry.

Publishing: after the alarm checks, write each Alm:* value and :Msg, then Sts:Banner and Sts:WorstSevr.

8.15 IOC start (restart)

- **At first tick after iocInit**, with parameters restored by autosave:
 1. Wait until the O2 and Alicat PVs are connected (log the wait), or 30 s have passed; treat still-missing PVs as invalid.
 2. valid = O2 severity < INVALID and $\text{o2Min} \leq \text{o2} \leq \text{o2Max}$.
 3. Log IOC started (autosaved settings restored).
 4. **Decide the state:**
 - if valid and $\text{o2} < \text{lidLevel}$ and $\text{Setpoint_RBV} > 0$: lastCmd = Setpoint_RBV, enter REGULATE (restart: O2 valid and setpoint > 0, resume regulation (§4.7)), configEpid, FBON = 1. epid starts bumplessly from the current setpoint.
 - else if $\text{Setpoint_RBV} \leq 0$: enter IDLE (restart: setpoint is 0 (§4.7))
 - else if not valid: enter OPEN_LOOP (restart: O2 invalid (§4.7))
 - else: enter IDLE (restart: O2 above lid threshold (§4.7))
- **Not restored** (they restart from zero): alarms, debounce state, histories (O2, rate, average, OVAL), hold-monitor counters, settling state.
- **Restored:** all parameters and mode slots, Mode, target, and the helium state (CumL, LastTotal, CylBase, the usage log, ledger and snapshots), plus OverrideCount.

- **Production note:** procServ runs `--noautorestart`, and cron starts IOCs at boot. A crash means a manual restart; the Alicat holds its last setpoint meanwhile.

8.16 Cylinder litres and run-out forecast (`newCylinder`, `cylForecast`)

- **Every tick:**
 - if `CylBase` is unset: `CylBase = Total_RBV` (first ever start: assume a full cylinder)
 - **Totalizer went backwards:** if `Total_RBV < CylBase - 1`, log MINOR Alicat totalizer went backwards (reset?): usage re-baselined, and set `CylBase = Total_RBV - (last logged used, or 0)`
 - `used = Total_RBV - CylBase`; `LeftL = max(0, cylCapacityL - used)`
- **Every 60 s** (`now mod 60 == 0`):
 1. Append (`now`, `used`) to the usage log, keeping 4 days.
 2. `span = now - time of the oldest entry`.
 3. **For each window $w \in \{3, 2, 1, 0.5, 0.25\}$ days:**
 - if `span < 0.9-w`: no estimate (not enough data yet)
 - otherwise: least-squares slope of `used` against `t` over the entries in the window gives rate (L/s)
 - if `LeftL ≤ 0`: estimate 0 h
 - else if `rate ≤ 1e-6`: no estimate (no usage)
 - else: `estimate = LeftL / rate / 3600 h`
 - publish `He:Est<W>H` and `He:Rate<W>` (L/day)
 4. **Median:** of the available estimates, the lower median (index $\lfloor (n-1)/2 \rfloor$ of the sorted list). Publish Min, Max, Median and `ForecastText`.
 5. **Alarm:** `median < cylAlarmH` → `Alm:CylLow = 2`, helium cylinder empty in `< <cylAlarmH> h` (forecast); `median < cylWarnH` → 1 (same text with `cylWarnH`); otherwise clear.
- **Cmd:NewCylinder:**
 - ledger event `cylinder`
 - `CylBase = Total_RBV`; clear the usage log
 - clear `CylLow` silently
 - log operator: `new He cylinder fitted (<cap> L usable)`; usage counted from `Total_RBV = <..> L`
- **Validation:** in the reference, after 4 days of use every window predicted 5.50–5.53 d, and the actual run-out was 5.52 d.

8.17 Helium ledger and usage report (`ledgerEvent`, `ledgerUpdate`, `litresAt`, `usageReport`)

- **Every tick:**
 - `CumL += max(0, Total_RBV - LastTotal)` (if `LastTotal` is known); `LastTotal = Total_RBV`
 - snapshot (`now`, `CumL`) when `now mod 3600 == 0` or when there are no snapshots yet
 - hourly: drop events and snapshots older than (`reportDays + 10`) days
- **Events:** (`t`, `type`, `CumL`) for purge (PURGE entry), zero (FLOW_ZERO or OPEN_STOP entry), cylinder (`Cmd:NewCylinder`) and newrun (`Cmd:MarkNewRun`).
- **litresAt(t):** interpolate linearly between the bracketing snapshots. If $t \geq \text{now}$, `CumL`; if $t \leq$ the first snapshot, the first snapshot's value.
- **Report**, recomputed every 60 s and after any ledger event:
 1. **Group purges into runs**, in time order:
 - a new run starts when (the gap from the **segment end** of the previous purge to this purge) $>$ `runGap`, or a `newrun` mark lies in (prev purge, this purge]
 - **segment end** = the time of the first `zero` event after the purge and before this purge (or the purge time, if there is none)
 2. **For each run:**
 - `end` = the first `zero` after its last purge (and before the next purge)

- finished = a next purge exists, or `now` – (zero time, or else last-purge time) > `runGap`, or a `newrun` mark exists after the last purge
 - if there is no zero and the run is finished, end = the last purge
 - otherwise the run is open (RunEnd = 0)
 - litres = (end ? end.L : CumL) – first purge's L
 - cylinders = the count of `cylinder` events in [start, end or now]
3. **Window:** wEnd = the latest run's end (or `now` if it is open or there are no runs); wStart = wEnd – `reportDays` · 86400.
4. **Totals:**
- Dispensed = `litresAt(wEnd)` – `litresAt(wStart)`
 - Cylinders = the count of `cylinder` events in (wStart, wEnd]
 - CylEquiv = Dispensed / `cylCapacityL`
5. **Runs in the window:** those with (end or `now`) > wStart. **InRuns** = Σ over them of (end ≤ wEnd ? end.L : `litresAt(wEnd)`) – (start ≥ wStart ? start.L : `litresAt(wStart)`). This uses exact event values at run boundaries; interpolation only where the window cuts a run.
6. **Publish** `He:Rep:*` and `He:Rep:Text`, formatted as the reference's Admin usage text.
- **Validation (reference scenario 19):**
 - user A across a 48 h downtime is one run (23 purges)
 - a marked changeover splits the runs
 - a run after 3.5 idle days splits automatically
 - Dispensed = Σ runs = the totalizer to within 1 L

8.18 Loss of Alicat connection

- **If the Alicat PVs are disconnected**, or `Flow_RBV` is INVALID, for more than `holdDetect` s in a state where the controller owns the flow:
 - raise `Alm:Mismatch` = 2 with text MFC not responding (CA disconnected)
 - continue the state machine without writing (puts would fail)
 - on reconnect: re-send `lastCmd` as in §8.7, and log
- **This is not in the reference**, whose simulated Alicat never disconnects. Keep it minimal.

8.19 Logging texts

- **Use the reference's message texts verbatim** (they are the acceptance-test oracle, §14).
- **Log line format:** `YYYY-MM-DD hh:mm:ss <STN> [MINOR|MAJOR] <text>`.

8.20 Write enable and shadow mode (NEW; not in the simulator; approved by the user 2026-09-25)

- `Par:writeEnable` (bo, autosaved). **Default 0 for the first production deployment.**
- **When 0, shadow mode:** the controller runs every rule, but:
 - no puts to `Setpoint`, `RampRate` or `Run`
 - `epid OUTL` is redirected to a local soft record `$(P)Shadow:Setpoint`. Implement this by having the SNL switch `PID.OUTL` between the two links, or by pointing `OUTL` at `$(P)Out:Setpoint`, a soft ao whose `FLNK` forwards only when enabled.
 - log shadow mode: would write `<PV> = <v>` for each suppressed write
- **Purpose:** commission and compare against manual operation without touching the Alicat. This honours the no-write rule until the user enables writes.
- **When 1:** normal operation. A transition 0 → 1 logs `writes enabled` and enters IDLE; the operator then decides (Purge, or leave it).
- **The bench runs with writes enabled** (the bench Alicat is simulated).

9. Parameter reference (defaults and limits)

- **Every row is a \$(P)Par:<key> record** with `DRVL` = min and `DRVH` = max.
- **"Level"** is the screen it appears on: U = main panel, A = Admin, D = Deep admin.
- **All values** are the reference defaults at `sim-v1.0`.

9.1 Station parameters

key	default	unit	min	max	level	meaning
target	0.99	%	0.2	5	U	O2 target
tol	0.02	%	0.005	0.5	A	In-range tolerance (display, alarms; not a deadband)
delta	-0.04	%	-1	1	A	Handoff margin below target (negative = above)
purgeFlow	20	SLPM	1	20	A	Purge flow
purgeTimeoutMargin	1.3	×	1	3	A	Timeout = margin × kinetic time
cylWarnH	24	h	1	240	A	Forecast MINOR below
cylAlarmH	6	h	0.5	120	A	Forecast MAJOR below
fbDelay	0	s	0	600	A	Delay after handoff flow settles, before FBON
flowMinorX	1.5	×exp	1.05	5	A	Flow-high MINOR
flowMajorX	2.0	×exp	1.1	10	A	Flow-high MAJOR
pinnedTime	600	s	30	7200	A	Pinned at max → MAJOR after
o2AbnormalOffset	0.3	%	0.02	5	A	O2 abnormally high above target by
settleTimeout	7200	s	600	86400	A	Target not reached within
V	41	L	5	200	D	Enclosure volume (lid-on decay rate F/V)
lidOnsetFrac	0.02	–	0.005	0.2	D	Decay onset = relative drop
lidOnsetMax	30	s	10	180	D	No onset within → open
lidWindow	15	s	4	60	D	Decay measured over
lidCurvMin	0.8	–	0.1	1	D	Open if 2nd/1st-half rate below
openSlopeFrac	0.5	–	0.05	0.95	D	Open if decay < fraction of F/V
dropSkipLevel	18	%	1	21	D	Lid check skipped if purge starts below
cylCapacityL	8000	L	100	50000	D	Usable He per full cylinder
reportDays	60	days	1	365	D	Usage report window
runGap	216000	s	3600	2592000	D	User run closes after no purge for (60 h)
handoffHold	5	s	1	120	D	Lag-corrected O2 below target-Δ for
purgeLag	12	s	0	60	D	Analyzer lag at purge flow (0 = none)
handoffFlowTol	0.05	SLPM	0.01	1	D	Handoff flow-settled tolerance
lidLevel	10	%	2	19	D	Lid detector O2 level
lidFilter	5	s	1	60	D	...above level for
lidSlope	0.2	%/s	0.005	2	D	...rise rate (10 s difference)
lidSlopeWindow	60	s	10	300	D	...rate look-back window
lidArmLevel	9	%	1	18	D	Detector arms in PURGE below

key	default	unit	min	max	level	meaning
rampMaxPurge	5	SLPM/s	0.5	20	D	Ramp clamp at purge start (0 counts as instant)
rampMinHandoff	2	SLPM/s	0.1	10	D	Ramp clamp at handoff (min)
hardCeiling	2.0	SLPM	0.1	20	D	Hard PID flow ceiling (surface vibration)
purgeTimeoutMin	120	s	30	1800	D	Purge timeout clamp
purgeTimeoutMax	1800	s	60	7200	D	Purge timeout clamp
ambientRef	19.4	%	5	21	D	O2 assumed at blind-purge start
flowLowX	0.5	×exp	0.05	0.95	D	Flow-low (wrong mode?) threshold
mismatchAbs	0.05	SLPM	0.01	2	D	Mismatch tolerance (absolute)
mismatchFrac	0.05	–	0	0.5	D	Mismatch tolerance (fraction)
mismatchMargin	5	s	1	120	D	Mismatch time margin after ramp
holdDetect	3	s	1	60	D	Hold monitor: on hold for
holdRetries	3	–	1	20	D	Fast Run attempts
holdRetryInterval	10	s	2	120	D	Fast retry interval
holdSlowRetry	60	s	10	3600	D	Slow retry interval
frozenTime	30	s	5	600	D	O2 frozen after unchanged for
o2Min	–0.5	%	–5	1	D	O2 plausible minimum
o2Max	25	%	19	100	D	O2 plausible maximum
avgN	10	samples	1	120	D	O2 average for epid
pidScan	10	s	1	120	D	PID step period
odel	0.01	SLPM	0	0.5	D	epid ODEL
gainSchedule	1	–	0	1	D	KP × 0.99/target
fineBand	0.01	%	0	0.5	D	Fine band (0 = off); ON by user decision
fineKPx	0.5	×	0	2	D	Fine band KP multiplier
fineKlx	1.0	×	0	2	D	Fine band KI multiplier
alarmDelay	10	s	0	300	D	Default alarm persistence
flowAlarmDelay	300	s	0	3600	D	Flow-high/low and O2-range persistence
progressMin	0.0005	%/min	0.0001	0.1	D	Settling progress threshold
stallGrace	300	s	0	3600	D	Progress judged after
flowSteadyBand	0.02	SLPM	0.001	1	D	PID output steady within ±
o2SteadyRate	0.002	%/min	0.0001	0.1	D	O2 steady below
stallWindow	120	s	20	600	D	Slope window
slopeAvgN	20	samples	1	60	D	Samples averaged at each end of the window
stallTime	120	s	10	1800	D	No progress this long → stalled
writeEnable	0	–	0	1	A	NEW: 0 = shadow mode (\$8.20)

9.2 Mode-slot limits

field	unit	min	max
baseFlow	SLPM	0.01	20
n	–	0.1	3

field	unit	min	max
KP	SLPM/%	-100	0
KI	1/s	0	0.05
drvh	SLPM	0.1	20
drvI	SLPM	0	5

- **Invariant:** the effective DRV L $\leq$ DRV H (§8.11 enforces it).
- **KP must be negative** (more flow lowers O₂).

10. Persistence (autosave)

- **sampleGas_settings.req:**
 - all Par:* (VAL)
 - all Mode:* fields
 - Mode (VAL) and the Mode mbbo state strings
 - Diag:OverrideCount
 - saved on change (monitor set, 30 s)
- **sampleGas_helium.req:**
 - He:CumL, He:LastTotal, He:CylBase
 - the arrays He:HistT/HistUsed/HistN, He:EvT/EvType/EvL/EvN, He:SnapT/SnapL/SnapN
 - saved every 300 s, and also written by the SNL (e.g. through a Cmd:SaveHelium trigger or autosave's manual_save) right after NewCylinder and MarkNewRun
- **Restore:** in pass 0, before iocInit. Arrays must restore completely; verify that autosave handles these sizes on the bench.
- **Bounded loss:** losing up to 5 min of helium log on a crash is accepted.

11. Production environment and conventions

11.1 Host and layout

- **Linux soft-IOC host,** under a dedicated EPICS service account.
- **synApps tree:** \$SUPPORT/, the production synApps support/ directory (ask the beamline staff for the path).
- **Custom IOCs** live under support/ChemMat/iocBoot/iocX/ with a startX script, e.g. iocSensors/startSensors.

11.2 Versions

- **Known:** asyn R4-44-2, calc R3-7-5, StreamDevice 2-8-24, seq (sequencer-mirror) R2-2-9. Base is at /usr/local/epics/base, version unknown.
- **Unknown:** std and autosave. Ask the user for 1s of the support tree and CONFIG_BASE_VERSION before the production build.

11.3 start_ioc entry to add

```
15LSS_sample_gas    20125    1    $SUPPORT/ChemMat/iocBoot/iocLSS_sample_gas/startLSSSampleGas
```

- **procServ template:** `procServ --noautorestart --logstamp -n "15LSS_sample_gas_IOC" -L <log_folder>15LSS_sample_gas.log 20125 <cmd>.`
- **Log folder:** `$IOC_LOGS/`, the beamline's shared IOC log folder.
- **Boot:** IOCs start via cron at reboot (`start_ioc -b`).

11.4 Existing screens and IOCs

- Existing screens and IOCs must remain untouched.
- The Alicat IOC `15Alicat` runs `ip-R2-22/iocs/ipExample/iocBoot/iocIpExample/Alicat.cmd`.

12. Recommended implementation structure (guidance, not normative)

- **SNL program** `sampleGas.st`, `reentrant (option +r;)`, with:
 - one state set `ctl`: a tick loop state that performs §8.1 by calling C functions, and holds the controller state as a variable
 - optionally a second state set for command handling
- **Why a loop state:** it avoids duplicating the tick ordering across SNL states. The controller "states" of §8.4 are data, not SNL states. This mirrors the reference exactly and keeps the ordering deterministic.
- **C library** `sampleGasLib.c`, pure functions and no CA:
 - slope from a ring buffer
 - lid-check decision
 - purge timeout
 - least-squares usage rate and forecast
 - ledger grouping and report
 - the debounce helper
 - fixed-size ring buffers; no heap allocation after init
 - unit tests with golden vectors from the reference (§14.4)
- **PV access:** `assign/monitor` for inputs. `pvPut(..., SYNC)` for the Alicat writes (so a failure is seen), and `put-callback` on `PID.PROC`.
- **Strings:** build messages with `epicsSnprintf`, using the exact texts of §8.
- **Sizes and time:** use the §7 array sizes; timestamps as epoch doubles.
- **Maintainability:** keep each helper unit-testable; one C file per concern is fine.

13. Phoebus screens (deliverable)

Three `.bob` displays, macro `P`, mirroring the simulator's operator column, Admin and Deep admin (see the screenshots in `docs/simulator/img/`).

13.1 Main panel (`sampleGas_main.bob`)

- **Banner:** `Sts:Banner`, coloured by `Sts:WorstSevr`.
- **Four big readouts:**
 - `Sts:02`, with target $\pm$ tol
 - `$(MFC)Flow_RBV`, with `Sts:ExpectedFlow` and the Alicat hold status
 - `$(MFC)Setpoint_RBV`, with `Sts:LastCmd`
 - `He:LeftL`, with `He:ForecastText`
- **State:** `Sts:State` as a coloured label, `Sts:StateDesc`, the `Sts:InRange` LED, and `Sts:Progress`.
- **Controls:**

- Mode combo; Par:target entry
- buttons Purge, Flow Zero, Resume PID (enabled only in OPEN_LOOP with Sts:02Valid), and New He cylinder fitted
- Admin (opens the admin display)
- **Also:** Sts:LastAction, and a trend of O2 and flow (log axes, target line).
- **Shadow mode:** a prominent "SHADOW MODE: no writes" badge when Sts:WriteEnable = 0.

13.2 Admin (sampleGas_admin.bob)

- **Parameters:** the A-level parameters, and the per-mode KP, KI, drvh and drvl for A–D.
- **Controls:** Par:writeEnable (with a confirm dialog), Release control, Mark new user run, Reset override count.
- **Displays:** Diag:OverrideLog, He:Rep:Text, the forecast windows, the epid internals and the key Diag: values. A button opens Deep admin.

13.3 Deep admin (sampleGas_deep.bob)

- The D-level parameters.
- Mode names, baseFlow and n.
- The linked PV names (macros, read-only).

13.4 Alarm server and heartbeat

- **Alarm server config** covering the Alm:* PVs, plus a disconnect or stale alarm on Sts:Heartbeat (stale if unchanged for more than 10 s). This is the only way an IOC crash is announced.
- **Archiving:** archive Sts:02, Sts:State, \$(MFC)Flow_RBV, \$(MFC)Setpoint_RBV and Sts:LastCmd at ≤ 10 s (1 s preferred), and the Alm:* on change. Configuring the archiver is up to the user; provide the PV list.

14. Testing and acceptance

14.1 Bench plant simulator (deliverable: ioc/test/plant_sim.py, caproto)

- **Purpose:** a PV-level port of the reference Plant class (§17). It serves, on localhost only:
 - SIM:Alicat1: with **Alicat_BC.db semantics**:
 - Setpoint: a put while Running_RBV = 0 changes VAL but not the device setpoint
 - Setpoint_RBV, Flow_RBV (1 Hz, 0.01 quantisation), Total_RBV, Running_RBV
 - RampRate/RampRate_RBV
 - Run: cancels hold; the device keeps its old setpoint
 - Gas_RBV (He = 7), FlowUnits_RBV = SLPM
 - SIM:02 (1 Hz; INVALID severity in invalid mode; value held in frozen mode)
- **Physics:** exactly as the reference Plant.step:
 - DT 0.25 s
 - well-mixed volume, ingress per lid with the flow-off blend
 - sensor-zone lag; transport delay via a history buffer whose delay grows at ≤ 0.5 s/s
 - the slow reading component (OU) and white noise
 - Alicat ramp and response τ 0.5 s; cylinder capacity limit
 - all parameters from defaultPlantParams()
- **World controls** as PVs SIM:World*:
 - LiftLid, CloseLid, CrackLid, LidType, Reseat
 - Hold (current, open, stuck), ClearHold

- SetRamp, Gas, CylPressure, AnalyzerMode, BreathDip
- Preset (regulating at the mode's expected flow, as `presetRegulating`)
- **Also:** a `SIM:World:Seed` for reproducible noise, and a switch to disable noise for deterministic comparisons.
- **Validation:** before using it for acceptance, drive it open-loop with a recorded flow sequence exported from the reference simulator (same seed, noise off), and match the O2 trace within 0.5 % relative.

14.2 Scenario acceptance (deliverable: `ioc/test/test_scenarios.py`, `pyepics`)

- **How it runs:** each of the reference's 19 scenarios, driven through `SIM:World:*` and the controller's `Cmd:*`, against the real IOC on the bench, in real time.
- **Pass criteria:** the expected end state, and log entries matching the regexes below (the reference `SELFTEST` table, `docs/simulator/`, and the self-test in the Help page).

#	Duration	End state	Required log patterns / checks
1	1 h	REGULATE	<code>lid check passed</code> , → REGULATE
2	10 min	OPEN_STOP	<code>enclosure open\?</code>
3	20 min	OPEN_STOP	<code>enclosure opened</code> , flow stopped
4	30 min	REGULATE	<code>lid check skipped</code>
5	30 min	REGULATE	<code>lid check passed</code>
6	4 h	REGULATE	<code>flow ≥ 2× expected</code>
7	1 h	REGULATE	<code>flow mismatch</code>
8	20 min	REGULATE	<code>MFC was on hold</code> , resumed
9	20 min	REGULATE	<code>MFC on hold</code> , cannot resume
10	40 min	REGULATE	<code>O2 reading frozen</code> , operator pressed Resume PID
11	20 min	REGULATE	<code>blind purge</code> , operator pressed Resume PID
12	20 min	REGULATE	<code>ramp rate 0</code> , handoff minimum
13	20 min	REGULATE	<code>flow mismatch</code>
14	20 min	REGULATE	<code>IOC crashed</code> is replaced by a real IOC kill and restart (§14.3); then resume regulation
15	1 h	REGULATE	<code>PID pinned at max flow</code> or <code>O2 abnormally high</code>
16	3 h	REGULATE	<code>settled: O2 reached 0.500</code>
17	(2 d)	–	≥ 3 forecast windows available (use §14.4 instead of real time)
18	1 h	REGULATE	<code>lid check passed</code>
19	(9 d)	–	2 runs, the first with 23 purges (use §14.4 instead of real time)

- **Scenario scripts:** the event times and actions are the reference `SCENARIOS` list. Read the `events` arrays.
- **Quantitative comparison:** for scenarios 1, 2, 3 and 8, with noise off and the same seed, compare the IOC trace with the reference simulator:
 - state-transition times within ±3 s
 - flows at transitions within ±0.02 SLPM
 - lid-check ratio within ±0.05
- **Runtime:** the full real-time suite takes ~14 h. Run it unattended on the bench.

14.3 IOC restart test

- Kill the IOC process during REGULATE, wait 5 min (the plant keeps running; the Alicat holds its flow), then restart it.
- **Expect:** REGULATE resumed, no setpoint bump larger than ± 0.02 SLPM in the first PID step, parameters and helium state restored, and a stale-heartbeat alarm raised by the client while it was down.

14.4 Algorithm unit tests (golden vectors)

- **Scope:** for each C helper, compare against the reference JavaScript on the same inputs:
 - slope, lid check, purge timeout, debounce
 - usage-rate fit and forecast
 - ledger grouping, `litresAt`, report
- **Generating vectors:** run the simulator headlessly, for example with Edge `--headless --dump-dom` on a small harness page that loads the simulator and calls `window.SIM` and the `Controller` methods. Save JSON inputs and outputs under `ioc/test/golden/`.
- **Scenarios 17 and 19** are accepted at this level: feed the ledger and forecast functions the event and totalizer sequences of those scenarios, and check the same outcomes as the reference self-test.

14.5 Definition of done (bench)

- The IOC builds on the MinGW bench with the traps of §2.3 handled.
- Unit tests pass. Scenario acceptance passes 17/17 real-time scenarios, and 17 and 19 pass by golden vectors.
- The restart test passes. Shadow mode is verified: no puts reach `SIM:Alicat1: while writeEnable = 0`.
- Screens render against the bench IOC.
- The simulator self-test still passes 19/19.
- All work is committed. A short `ioc/README.md` covers how to build, run the bench and run the tests.

15. Deployment and commissioning (with the user)

1. **Production build** on the Linux host, after getting the module versions (§11.2). Install it under `ChemMat/iocBoot/iocLSS_sample_gas/`. Add the `start_ioc` line.
 2. **Shadow mode** (`writeEnable = 0`). Run alongside the existing timer script and manual operation for at least one user run. Compare what the controller *would* have done:
 - purge handoffs
 - lid-open detections (compare them with the operators' actions)
 - the flows the PID would command
 - the forecast against the actual cylinder changes
 - Fix discrepancies before enabling writes.
 3. **Supervised hardware tests** (user present; `writeEnable = 1`; only once the user lifts the no-write rule):
 - **Open-lid purge:** confirm the lid check's rate and curvature thresholds on the real box.
 - **Surface-vibration ceiling test:** set `hardCeiling` and the mode `drvh`.
 - **Bump tests per mode** (design spec §6.1): confirm KP and KI.
 - **Low-flow test on the normal lid:** confirm the flow-off blend.
 - **Collimator-lid 1 Hz record:** noise and lid kinetics.
 4. **Production:** set `writeEnable = 1`, retire the timer script, and record every parameter change in the design spec.
-

16. Open items (the implementer must not guess these)

Item	Owner	Blocking for
15IDE PV names (MFC prefix, O2 PV)	user	15IDE station only
Production base, std and autosave versions	user	production build
Real usable litres per helium cylinder (cylCapacityL)	user	forecast accuracy
Cylinder-pressure PV	user (future)	nothing (optional)
Lifting the no-write rule on 15IDC:*	user	\$15.3 onwards
Hardware test results (lid check on an open box, ceiling, bump tests)	user + agent	final parameters

17. Traceability

IOC spec	Design spec	Reference (simulator/sample_gas_simulator.html, tag sim-v1.0)
§8.1 tick	\$4	Controller.tick, Station.step
§8.2 inputs	\$4.6, \$5	readInputs
§8.3 command	\$5.1	command
§8.4 states	\$4	enter, doHandoff, tick switch
§8.5 commands	\$7	opPurge, opFlowZero, opResume, opIdle, setTarget
§8.6 lid detector	\$4.6, \$4.6.1	lidDetector
§8.7 hold monitor	\$4.6.2	holdMonitor
§8.8 precheck	\$4.1	doPrecheck
§8.9–8.10 purge	\$4.2, \$4.3	doPurge, purgeTimeout
§8.11 PID	\$6	configEpid, processEpid
§8.12 settling	\$5.1.1	startSettling, alarmChecks
§8.13 expected flow	\$5.2	expectedFlow
§8.14 alarms	\$5, \$5.1, \$5.1.1	alarmChecks, setAlarm, debounce, latch, override
§8.15 IOC start	\$4.7	restart
§8.16 cylinder	\$5.5	newCylinder, cylForecast
§8.17 ledger	\$5.6	ledgerEvent, ledgerUpdate, litresAt, usageReport
§9 parameters	\$7.0, \$9	defaultControllerParams, ADMIN_FIELDS, DEEP_FIELDS, mode tables
§14.1 plant	\$2.1, \$8	defaultPlantParams, Plant
§14.2 scenarios	\$8	SCENARIOS, SELFTEST